

▼ Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
netflix_dataset=pd.read_csv('/content/drive/MyDrive/netflix_recommendation_system(SVD)/Copy of combined_data_1.txt.zip',header=
```

```
netflix_dataset
```

	Cust_Id	Ratings
0	1:	NaN
1	1488844	3.0
2	822109	5.0
3	885013	4.0
4	30878	4.0
...	...	...
24058258	2591364	2.0
24058259	1791000	2.0
24058260	512536	5.0
24058261	988963	3.0
24058262	1704416	3.0

24058263 rows × 2 columns

▼ EDA

```
netflix_dataset.isnull().sum()
```

	0
Cust_Id	0
Ratings	4499

dtype: int64

```
movie_count=netflix_dataset.isnull().sum()
movie_count=movie_count['Ratings']
movie_count
```

4499

```
total_count=netflix_dataset['Cust_Id'].nunique()
```

```
customer_count=total_count-movie_count
customer_count
```

470758

```
ratings_count=netflix_dataset['Cust_Id'].count()-movie_count
ratings_count
```

24053764

```
Stars=netflix_dataset['Ratings'].value_counts()
```

Stars

	count
Ratings	
4.0	8085741
3.0	6904181
5.0	5506583
2.0	2439073
1.0	1118186

dtype: int64

```

movie_id=None
movie_np=[]

for customer in netflix_dataset['Cust_Id']:
    if ":" in customer:
        movie_id=int(customer.replace(":", "")) #1:-->1
        movie_np.append(movie_id)

```

```
netflix_dataset['Movie_id']=movie_np
```

netflix_dataset

	Cust_Id	Ratings	Movie_id
0	1:	NaN	1
1	1488844	3.0	1
2	822109	5.0	1
3	885013	4.0	1
4	30878	4.0	1
...	...	...	...
24058258	2591364	2.0	4499
24058259	1791000	2.0	4499
24058260	512536	5.0	4499
24058261	988963	3.0	4499
24058262	1704416	3.0	4499

24058263 rows × 3 columns

```
netflix_dataset.dropna(inplace=True)
```

netflix_dataset

	Cust_Id	Ratings	Movie_id
1	1488844	3.0	1
2	822109	5.0	1
3	885013	4.0	1
4	30878	4.0	1
5	823519	3.0	1
...	...	...	...
24058258	2591364	2.0	4499
24058259	1791000	2.0	4499
24058260	512536	5.0	4499
24058261	988963	3.0	4499
24058262	1704416	3.0	4499

24053764 rows × 3 columns

```
netflix_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 24053764 entries, 1 to 24058262
Data columns (total 3 columns):
#   Column      Dtype
---  ---
0    Cust_Id    object
1    Ratings    float64
2    Movie_id   int64
dtypes: float64(1), int64(1), object(1)
memory usage: 734.1+ MB
```

```
netflix_dataset['Cust_Id']=netflix_dataset['Cust_Id'].astype(int)
```

```
netflix_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 24053764 entries, 1 to 24058262
Data columns (total 3 columns):
#   Column      Dtype
---  ---
0    Cust_Id    int64
1    Ratings    float64
2    Movie_id   int64
dtypes: float64(1), int64(2)
memory usage: 734.1 MB
```

```
movie_review_count=netflix_dataset['Movie_id'].value_counts()
movie_review_count
```

```

      count
Movie_id
1905    193941
2152    162597
3860    160454
4432    156183
571     154832
...
4294      44
915       43
3656      42
4338      39
4362      36

4499 rows × 1 columns

dtype: int64

```

```

bench_mark=round(movie_review_count.quantile(0.6),0)
bench_mark

```

```

908.0

```

```

drop_movie_index=movie_review_count[movie_review_count<bench_mark].index

```

```

drop_movie_index

```

```

Index([1598, 1733, 1647, 4099, 1616, 1446, 263, 4259, 160, 1988,
      ...
      1858, 4035, 3693, 2805, 820, 4294, 915, 3656, 4338, 4362],
      dtype='int64', name='Movie_id', length=2699)

```

```

len(drop_movie_index)

```

```

2699

```

```

movie_customer_count=netflix_dataset['Cust_Id'].value_counts()
movie_customer_count

```

```

      count
Cust_Id
305344    4467
387418    4422
2439493    4195
1664010    4019
2118461    3769
...
1300341      1
2550360      1
11848       1
930788      1
594210      1

470758 rows × 1 columns

dtype: int64

```

```
bench_mark_cust=round(movie_customer_count.quantile(0.6),0)
```

```
bench_mark_cust
```

```
36.0
```

```
drop_cust_index=movie_customer_count[movie_customer_count<bench_mark_cust].index
```

```
drop_cust_index
```

```
Index([2194851, 600295, 1739398, 1157368, 532108, 2157249, 256134, 640441,
       1272324, 1346990,
       ...,
       1969065, 899932, 611596, 2147176, 811650, 1300341, 2550360, 11848,
       930788, 594210],
      dtype='int64', name='Cust_Id', length=282042)
```

```
netflix_dataset=netflix_dataset[~netflix_dataset['Movie_id'].isin(drop_movie_index)]
netflix_dataset=netflix_dataset[~netflix_dataset['Cust_Id'].isin(drop_cust_index)]
```

```
netflix_dataset
```

	Cust_Id	Ratings	Movie_id
696	712664	5.0	3
697	1331154	4.0	3
698	2632461	3.0	3
699	44937	5.0	3
700	656399	4.0	3
...	...	...	...
24056842	1055714	5.0	4496
24056843	2643029	4.0	4496
24056844	267802	4.0	4496
24056845	1559566	3.0	4496
24056846	293198	3.0	4496

```
19695836 rows × 3 columns
```

Model Building

```
movie_titles=pd.read_csv("/content/drive/MyDrive/netflix_recommendation_system(SVD)/Copy of movie_titles.csv",encoding='ISO-885
```

```
movie_titles
```

	Movie_Id	Year	Name
0	1	2003.0	Dinosaur Planet
1	2	2004.0	Isle of Man TT 2004 Review
2	3	1997.0	Character
3	4	1994.0	Paula Abdul's Get Up & Dance
4	5	2004.0	The Rise and Fall of ECW
...	...	...	...
17765	17766	2002.0	Where the Wild Things Are and Other Maurice Se...
17766	17767	2004.0	Fidel Castro: American Experience
17767	17768	2000.0	Epoch
17768	17769	2003.0	The Company
17769	17770	2003.0	Alien Hunter

17770 rows × 3 columns

```
!pip install numpy==1.26.4
```

Requirement already satisfied: numpy==1.26.4 in /usr/local/lib/python3.12/dist-packages (1.26.4)

```
!pip install scikit-surprise
```

Requirement already satisfied: scikit-surprise in /usr/local/lib/python3.12/dist-packages (1.1.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-surprise) (1.5.2)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-surprise) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-surprise) (1.16.3)

Component	Purpose
<code>SVD</code>	Matrix Factorization algorithm — one of the most popular collaborative filtering models.
<code>Reader</code>	Defines the format of your rating data (e.g., rating scale 1–5, file type, column order).
<code>Dataset</code>	Loads your data into a Surprise-compatible structure.
<code>cross_validate</code>	Performs model evaluation using k-fold cross-validation (like in scikit-learn).

-fold cross-validation as an example (used in `cross_validate()`):

Split dataset into k equal parts (called folds). Example: For 5-fold CV → data is split into 5 subsets.

Perform k iterations of training/testing:

In each iteration:

Use k-1 folds for training.

Use 1 fold for testing.

Rotate the test fold each time.

Compute your evaluation metric (e.g., RMSE, MAE, Accuracy) for each fold.

Average all the metric values — that's your final performance score.

Fold	Training Data	Testing Data	RMSE (Example)
1	Folds 2–5	Fold 1	0.88
2	Folds 1,3–5	Fold 2	0.84
3	Folds 1–2,4–5	Fold 3	0.86
4	Folds 1–3,5	Fold 4	0.85
5	Folds 1–4	Fold 5	0.87

```
from surprise import SVD, Reader, Dataset, accuracy
from surprise.model_selection import cross_validate
```

```
reader=Reader()
```

```
data=Dataset.load_from_df(netflix_dataset[['Movie_id', 'Cust_Id', 'Ratings']][:10000], reader)
```

data

<surprise.dataset.DatasetAutoFolds at 0x7d87fa01f980>

model=SVD()

#Cross Validation

cross_validate(model,data,measures=['RMSE'],cv=3)

```
{'test_rmse': array([1.01869882, 1.01720978, 1.01975892]),
 'fit_time': (4.509923934936523, 2.6749088764190674, 2.192540168762207),
 'test_time': (0.7955763339996338, 0.2701447010040283, 0.6481847763061523)}
```

#select a single user for recommending

```
user_rating=netflix_dataset[netflix_dataset['Cust_Id']==1331154]
user_rating
```

	Cust_Id	Ratings	Movie_id
697	1331154	4.0	3
5178	1331154	4.0	8
31460	1331154	3.0	18
92840	1331154	4.0	30
224761	1331154	3.0	44
...	...	...	...
23439584	1331154	4.0	4389
23546489	1331154	2.0	4402
23649431	1331154	4.0	4432
23844441	1331154	3.0	4472
23985561	1331154	4.0	4488

253 rows × 3 columns

#creating the duplicate copy of movie_titles

```
user_1331154=movie_titles.copy()
user_1331154
```

	Movie_Id	Year	Name
0	1	2003.0	Dinosaur Planet
1	2	2004.0	Isle of Man TT 2004 Review
2	3	1997.0	Character
3	4	1994.0	Paula Abdul's Get Up & Dance
4	5	2004.0	The Rise and Fall of ECW
...	...	...	...
17765	17766	2002.0	Where the Wild Things Are and Other Maurice Se...
17766	17767	2004.0	Fidel Castro: American Experience
17767	17768	2000.0	Epoch
17768	17769	2003.0	The Company
17769	17770	2003.0	Alien Hunter

17770 rows × 3 columns

user_1331154=user_1331154[~ user_1331154['Movie_Id'].isin(drop_movie_index)]

user_1331154 #after removing the movies in drop_movie_index

	Movie_Id	Year	Name
2	3	1997.0	Character
4	5	2004.0	The Rise and Fall of ECW
5	6	1997.0	Sick
7	8	2004.0	What the #\$! Do We Know!?
15	16	1996.0	Screamers
...	...	...	...
17765	17766	2002.0	Where the Wild Things Are and Other Maurice Se...
17766	17767	2004.0	Fidel Castro: American Experience
17767	17768	2000.0	Epoch
17768	17769	2003.0	The Company
17769	17770	2003.0	Alien Hunter

15071 rows × 3 columns

```
user_1331154['Estimated']=user_1331154['Movie_Id'].apply(lambda x: model.predict(1331154,x).est)
```

/tmp/ipython-input-3617067989.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
user_1331154['Estimated']=user_1331154['Movie_Id'].apply(lambda x: model.predict(1331154,x).est)
```

user_1331154

	Movie_Id	Year	Name	Estimated
2	3	1997.0	Character	3.582612
4	5	2004.0	The Rise and Fall of ECW	3.582612
5	6	1997.0	Sick	3.582612
7	8	2004.0	What the #\$! Do We Know!?	3.582612
15	16	1996.0	Screamers	3.582612
...	...	...	...	...
17765	17766	2002.0	Where the Wild Things Are and Other Maurice Se...	3.582612
17766	17767	2004.0	Fidel Castro: American Experience	3.582612
17767	17768	2000.0	Epoch	3.582612
17768	17769	2003.0	The Company	3.582612
17769	17770	2003.0	Alien Hunter	3.582612

15071 rows × 4 columns


```
user_1331154.sort_values('Estimated',ascending=False)
```

	Movie_Id	Year	Name	Estimated
10373	10374	1997.0	Goosebumps: Scary House	3.851422
6459	6460	2005.0	The Game: The Documentary: The DVD	3.828524
17450	17451	2000.0	Along for the Ride	3.828301
5726	5727	1984.0	The Last Starfighter	3.792587
11042	11043	1970.0	Mary Tyler Moore: Season 1	3.775896
...	...	...	...	...
10191	10192	1969.0	Mosquito Squadron	3.360801
6177	6178	2001.0	Underwaterworld Trilogy: Deep Encouters / Ocea...	3.334474
1499	1500	1987.0	Miss Marple: Collection 2	3.331958
16153	16154	2003.0	Warren Miller's: The Power of Snow: Fifty	3.330142
6812	6813	2002.0	My Beautiful Girl Mari	3.303309

15071 rows × 4 columns

```
user_1331154.head()
```

	Movie_Id	Year	Name	Estimated
2	3	1997.0	Character	3.582612
4	5	2004.0	The Rise and Fall of ECW	3.582612
5	6	1997.0	Sick	3.582612
7	8	2004.0	What the #\$*! Do We Know!?	3.582612
15	16	1996.0	Screamers	3.582612